VNet-to-VNet VPN Implementation Using Azure VPN Gateway

Name: Kushagar

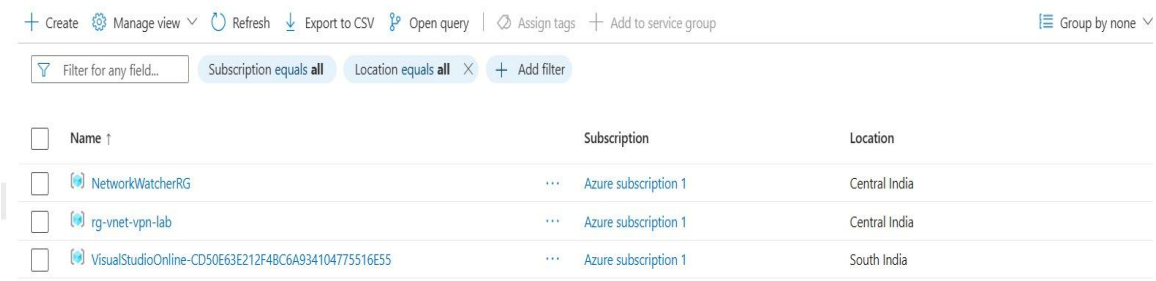
Objective

To create a secure connection between two Azure Virtual Networks using VPN Gateway and verify connectivity using virtual machines.

Detailed Implementation Steps

Step 1: Create Resource Group

Name: rg-vnet-vpn-lab
Region: Central India



Step 2: Create VNet1

Name: vnet1
Address space: 10.0.0.0/16
Subnet1: 10.0.1.0/24
GatewaySubnet: 10.0.255.0/27

Step 3: Create VNet2

Name: vnet2
Address space: 10.1.0.0/16
Subnet1: 10.1.1.0/24
GatewaySubnet: 10.1.255.0/27

Step 4: Create VPN Gateway for VNet1

Name: vnet1-gateway
Type: VPN
VPN Type: Route-based
SKU: VpnGw1
Public IP: pip-vnet1-gw
Active-active: Disabled
BGP: Disabled

Step 5: Create VPN Gateway for VNet2

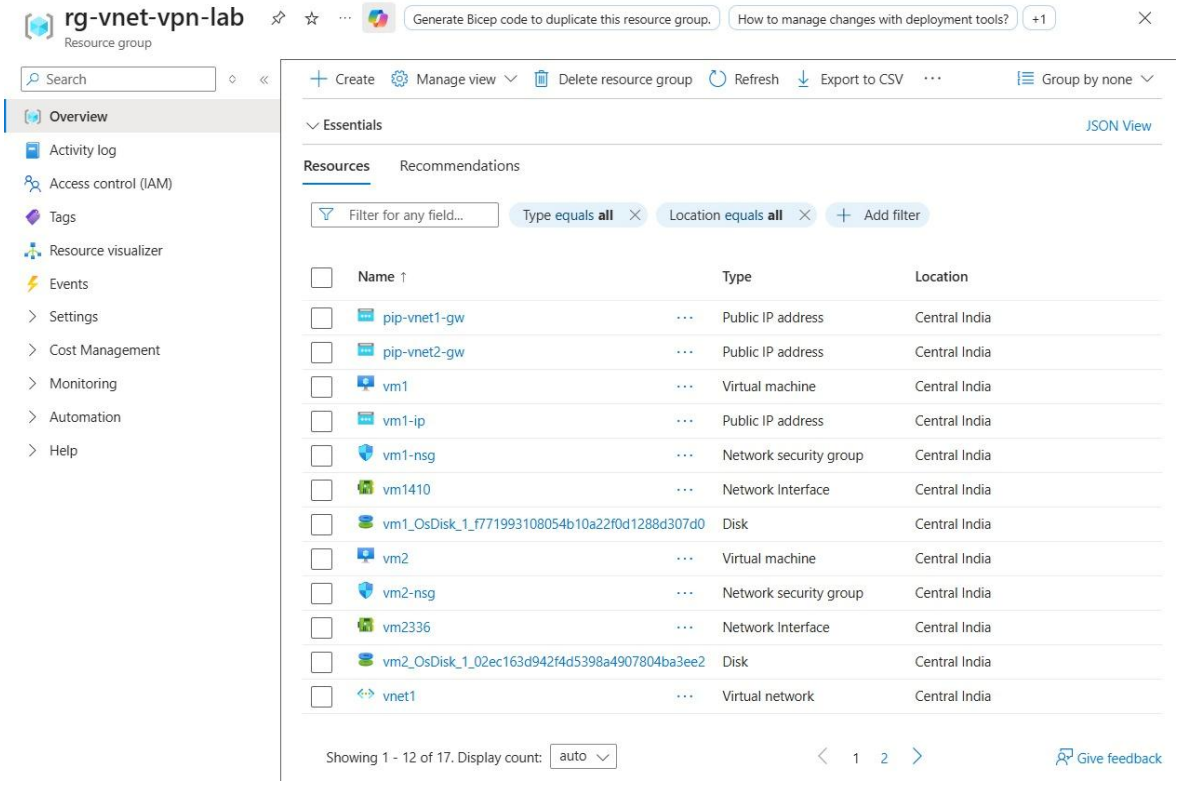
Name: vnet2-gateway
Public IP: pip-vnet2-gw
Same configuration as above

Step 6: Create Connection

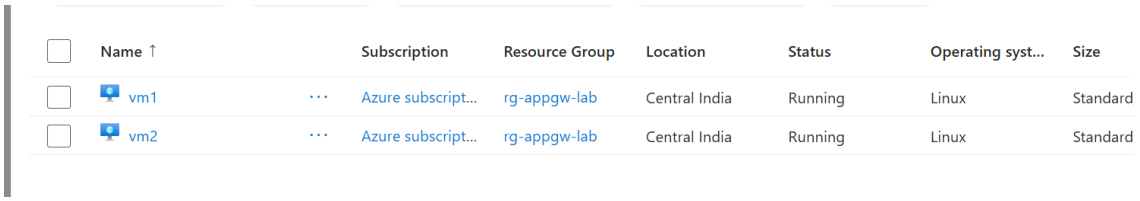
Connection Name: vnet1-to-vnet2
Type: VNet-to-VNet
Shared Key: Azure123

Step 7: Reverse Connection

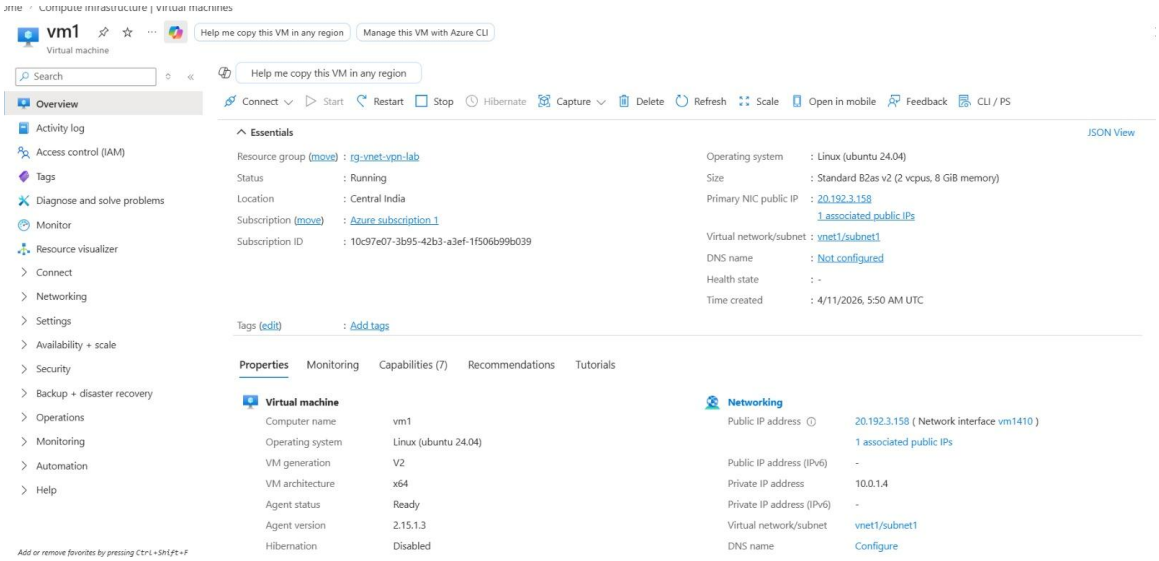
Connection Name: vnet2-to-vnet1
Shared Key: Azure123



Step 8: Create Virtual Machines

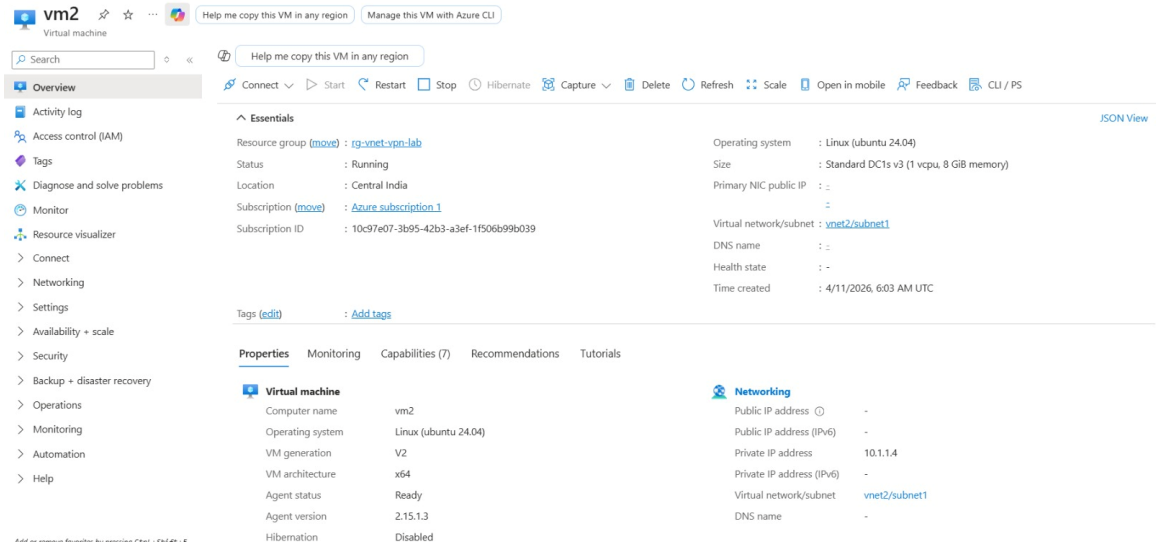


VM1:
Name: vm1
VNet: vnet1
Subnet: subnet1
Public IP: Enabled
Username: azureuser
Password: (user-defined)



VM2:
Name: vm2
VNet: vnet2
Subnet: subnet1
Public IP: None (due to free subscription limit)
Username: azureuser1
Password: (user-defined)

Note: VM2 was created without public IP because Azure free subscription allows limited public IPs.



Step 9: Connectivity Test

From VM1:
ping 10.1.1.4
From VM2:
ping 10.0.1.4

```
azureuser1@vm2: ~

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\malle> ssh azureuser@20.192.3.158
The authenticity of host '20.192.3.158 (20.192.3.158)' can't be established.
ED25519 key fingerprint is SHA256:l2EOk1BIvGWwxzm0+8ovv9dTSjnA16dUi8DRQWyEtjc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.192.3.158' (ED25519) to the list of known hosts.
azureuser@20.192.3.158's password:
client_global_hostkeys_prove_confirm: server gave bad signature for RSA key 0: incorrect signature
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1010-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Apr 11 06:09:35 UTC 2026

  System load:  0.08              Processes:             131
  Usage of /:   5.7% of 28.02GB   Users logged in:       0
  Memory usage: 4%                IPv4 address for eth0: 10.0.1.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
```

```
azureuser@vm1:~$ ssh azureuser1@10.1.1.4
azureuser1@10.1.1.4's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1010-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Apr 11 06:13:29 UTC 2026

  System load:  0.0               Processes:             125
  Usage of /:   5.7% of 28.02GB   Users logged in:       0
  Memory usage: 4%                IPv4 address for eth0: 10.1.1.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
azureuser1@vm2:~$ ping 10.0.1.4
PING 10.0.1.4 (10.0.1.4) 56(84) bytes of data.
64 bytes from 10.0.1.4: icmp_seq=1 ttl=64 time=1.68 ms
64 bytes from 10.0.1.4: icmp_seq=2 ttl=64 time=1.40 ms
64 bytes from 10.0.1.4: icmp_seq=3 ttl=64 time=2.26 ms
64 bytes from 10.0.1.4: icmp_seq=4 ttl=64 time=1.46 ms
64 bytes from 10.0.1.4: icmp_seq=5 ttl=64 time=1.50 ms
64 bytes from 10.0.1.4: icmp_seq=6 ttl=64 time=1.35 ms
64 bytes from 10.0.1.4: icmp_seq=7 ttl=64 time=1.39 ms
64 bytes from 10.0.1.4: icmp_seq=8 ttl=64 time=1.46 ms
64 bytes from 10.0.1.4: icmp_seq=9 ttl=64 time=1.75 ms
64 bytes from 10.0.1.4: icmp_seq=10 ttl=64 time=1.29 ms
64 bytes from 10.0.1.4: icmp_seq=11 ttl=64 time=1.48 ms
64 bytes from 10.0.1.4: icmp_seq=12 ttl=64 time=1.51 ms
64 bytes from 10.0.1.4: icmp_seq=13 ttl=64 time=1.34 ms
64 bytes from 10.0.1.4: icmp_seq=14 ttl=64 time=1.41 ms
64 bytes from 10.0.1.4: icmp_seq=15 ttl=64 time=1.51 ms
64 bytes from 10.0.1.4: icmp_seq=16 ttl=64 time=1.45 ms
64 bytes from 10.0.1.4: icmp_seq=17 ttl=64 time=1.66 ms
64 bytes from 10.0.1.4: icmp_seq=18 ttl=64 time=1.47 ms
64 bytes from 10.0.1.4: icmp_seq=19 ttl=64 time=1.60 ms
64 bytes from 10.0.1.4: icmp_seq=20 ttl=64 time=1.66 ms
64 bytes from 10.0.1.4: icmp_seq=21 ttl=64 time=1.57 ms
64 bytes from 10.0.1.4: icmp_seq=22 ttl=64 time=1.43 ms
64 bytes from 10.0.1.4: icmp_seq=23 ttl=64 time=2.10 ms
64 bytes from 10.0.1.4: icmp_seq=24 ttl=64 time=2.99 ms
64 bytes from 10.0.1.4: icmp_seq=25 ttl=64 time=1.44 ms
64 bytes from 10.0.1.4: icmp_seq=26 ttl=64 time=1.40 ms
64 bytes from 10.0.1.4: icmp_seq=27 ttl=64 time=1.47 ms
64 bytes from 10.0.1.4: icmp_seq=28 ttl=64 time=1.77 ms
64 bytes from 10.0.1.4: icmp_seq=29 ttl=64 time=1.45 ms
64 bytes from 10.0.1.4: icmp_seq=30 ttl=64 time=1.47 ms
64 bytes from 10.0.1.4: icmp_seq=31 ttl=64 time=2.81 ms
```

Conclusion

The VNet-to-VNet VPN was successfully created and verified using ping between virtual machines.